CHAPTER 5: WHEN A SECOND AGENT IS A LIABILITY

Summary: *Multi-agent designs get chosen for reasons drawn from somewhere other than the work itself. Team structure, headcount, a diagram that looks impressive to a steering committee. Only two properties of the task itself ever justify splitting it, and everything else is a story about the organisation. This chapter names those two properties, sets out what a second agent costs in guarantees you silently give up, and covers the machinery a real orchestration needs before it can be trusted.*

Two Properties of the Work That Justify a Second Agent

Start from the strict version, because it disposes of most wrong answers immediately.

Only **two things about a task** can show that a **single agent falls short**.

Irreconcilable specialisation. Two parts of the work need instructions, tools or constraints that genuinely cannot coexist. Not “would be tidier apart”. Contradictory. One part must refuse to speculate while the other must generate options; one needs write access to a system the other must never touch.

Real independence between sub-tasks. Parts of the work do not need each other’s results, so they can proceed at the same time without one waiting on the other.

Everything else is a fact about your organisation rather than about the task. Three teams own three systems. A department wants its own agent. The vendor’s reference architecture has five boxes. Each of these may be a real political constraint you have to live with, and **not one of them is evidence** that a **single agent falls short**. The distinction matters because architecture chosen from an org chart inherits the org chart’s problems and adds new ones.

What a Second Agent Costs

Splitting work does not just add coordination. It removes three guarantees that a single component quietly provided.

Ordering stops being a guarantee and becomes a behaviour. With one component owning the sequence, the order things happen in is a property of the design. Distribute the decision and the order becomes whatever emerges from the interaction, which is usually right and occasionally is not, and the occasions are not reproducible.

Halt-on-exception stops being a guarantee. A single owner of sequencing can stop everything the moment a step fails. Independent decision-makers cannot, because each one only knows about its own failure.

The **audit trail fragments**. The complete record of what happened in what order exists only where sequencing is owned. Split it and reconstructing a run becomes a correlation exercise across logs.

Name the thing this produces: the **uncatchable failure** is a subagent returning a confident, well-formed, wrong result that the orchestrator has no independent means of doubting. A single agent that goes wrong at least goes wrong in one place you can look at. A fleet of agents produces a well-formatted consensus that no human is accountable for any part of.

The **named anti-pattern is treating an org chart as an architecture diagram**. It is seductive because it makes ownership obvious and delivery political rather than technical.

The Orchestrator Owns Sequencing, or Nobody Does

The pattern that keeps the guarantees is the one where a **single component owns sequencing**, decides what runs next, and holds the rule about what happens when something fails. Workers do work. They do not decide the order, any more than the humans on a production line decide the sequence of stations.

That single owner is what converts ordering and halt-on-exception from emergent behaviour into design properties, and it is the same thing that makes the audit trail complete rather than distributed.

Anthropic publishes a worked orchestration example, and its details are worth knowing because they are the concrete answer to "what does a serious multi-agent design contain?"[1]

Concern	What the documented design does
One worker breaks	Failures are isolated, so a broken subagent degrades to an error string instead of ending the whole run
A connection drops	A per-request timeout bounds each call, so the damage is confined to one subagent
The run is interrupted	A content-addressed journal, keyed on a hash of each subagent's prompt, means a restart recomputes only the unfinished sub-tasks
A worker returns something plausible and wrong	A second wave of verifiers re-derives claims from source, and **defaults to refuted when uncertain**
Results come back as prose	Subagents report through a dedicated tool, so results arrive as structured JSON

Two of those rows deserve to be lifted out.

Defaulting to refuted when uncertain is the direct countermeasure to the uncatchable failure. A verifier that resolves ambiguity in favour of the claim adds a rubber stamp and a cost line. The default has to point the other way for verification to mean anything.

Reporting through a tool rather than in prose is the difference between a result you can act on and a result you have to parse. Anthropic makes the general form of this point elsewhere and it is worth carrying past this chapter: tool use is a contract in

which your application specifies the operations and the shape of their inputs and outputs, while the model decides when to call them, and writing a **regular expression to extract a decision** out of model output is a clear sign that the **decision should have been a tool call**.[2]

The Mechanics That Bite in Parallel Work

Three documented behaviours cause real defects in multi-agent and parallel-tool designs, and all three are invisible until they happen.

Every tool call needs a result, including the ones you **decline to run**. When several tool calls come back in one turn, one result must be returned for each, together in the next message, matched to its call by identifier. If you choose not to run one, it still needs a result, marked as an error with a brief explanation.[3] Silence is not a valid response to a call.

An **unfinished server tool call has no marker except a missing result**. A response can contain a server tool call whose identifier has no matching result block, which means that call did not finish. There is no other signal for this state, and the way to detect it is to check each call's identifier for a matching result.[4] A design that assumes every call in a response completed will silently drop work.

A **server-side loop can pause on an iteration limit**. When a server-side sampling loop reaches its limit, documented with a default of 10, the turn comes back paused, and the caller handles it by continuing the conversation.[4] This is a normal control-flow event, not an error, and treating it as a failure throws away completed work.

One budget-shaped trap belongs here too. Task budgets give the model a token allowance for a whole agentic loop, presented as a countdown it paces itself against, but the **budget is a soft hint rather than a hard cap**. The model may exceed it when stopping mid-action would be more disruptive than finishing, and the enforced ceiling on generation remains the request's maximum token setting.[5] If a **scenario needs a hard limit, a task budget is not** it.

Tool Confusion Scales With Unrelated Domains, Not With Effort

A specific failure shows up in agents that have accumulated capability: the agent picks the wrong tool, and picks it more often as the system grows.

The important property is what the symptom scales with. It grows with the number of unrelated domains sharing a single context, not with how the tools are described. So the durable fix reduces what any one agent has to choose among, by splitting along genuine domain boundaries.

Everything else leaves the crowded surface exactly as it was. Longer tool descriptions leave it intact. More examples leave it intact. Adding a routing instruction to the system prompt leaves it intact and adds a paragraph. Adding another capability to the same agent enlarges it.

This is one of the few cases where the answer really is another agent, and notice that it qualifies under the strict rule from the first section: the domains are genuinely separable work, not an org chart.

How Wrong Answers Are Built on Multi-Agent Questions

- **Organisational facts offered as architectural evidence**. The option cites team boundaries, ownership, headcount, or a desire for department-level autonomy. The test: does the reason describe the **task**, or the **company**? Only the task can establish that one agent is insufficient.

- **Coordination proposed where sequence must be guaranteed**. The option distributes a decision that has to happen in a fixed order or stop on failure. The test: is there a **single component that owns the order**? If not, ordering is **emergent**.

- **Instruction offered against a structural symptom**. The option answers tool confusion with better descriptions, more examples or a routing paragraph. The test: does this reduce the **number of unrelated domains** one agent chooses among?

- **Verification that cannot fail**. The option adds a review step that accepts when unsure. The test: which **way does it resolve ambiguity**? A checker that **defaults to approval** is a cost centre.

When the scenario says	The answer is usually	The trap is usually
Parts of the work need contradictory instructions or tool access	Split them; this is irreconcilable specialisation	Splitting because two parts are merely different
Sub-tasks genuinely do not need each other's results	Split them and run in parallel	Splitting work whose later steps consume earlier output
Teams, departments or ownership are given as the reason	One agent, unless the work itself demands otherwise	Accepting an organisational fact as architectural evidence
Order must be guaranteed, or everything must stop on a failure	One component owns sequencing	Peer agents coordinating, which makes ordering emergent
An agent picks the wrong tool more often as it grows	Reduce how many unrelated domains one agent chooses among	Longer descriptions, more examples, or a routing instruction
A subagent may return something plausible and wrong	A verification pass that re-derives from source and defaults to refuted	A review step that approves when uncertain
Several tool calls returned in one turn	One result for each, matched by identifier, including errors for calls you declined	Returning results only for the calls you ran

When the scenario says	The answer is usually	The trap is usually
A tool call has no matching result block	That call is unfinished; detect it by checking identifiers	Assuming every call in the response completed
A turn comes back paused at an iteration limit	Continue the conversation	Treating the pause as a failure and discarding the work
Spend must not exceed a hard limit	The request's maximum token setting	A task budget, which is a soft hint the model may exceed
Results come back as prose you have to parse	Have subagents report through a dedicated tool as structured output	Writing a regular expression against model output

The habit worth carrying: for **every proposed agent, name the guarantee it removes**. If you cannot name one, you have not understood the design well enough to add it.

What to Remember

- Only **two properties of the work** justify more than one agent: **irreconcilable specialisation**, meaning parts that need instructions, tools or constraints that cannot coexist, and **real independence**, meaning sub-tasks that do not consume each other's results. Team structure, ownership and headcount describe the organisation, and **no fact of that kind** can show a single agent falls short.

- Splitting work removes **three guarantees**. Ordering becomes **emergent rather than designed**, **halt-on-exception stops being enforceable** because each part only knows its own failure, and the **audit trail fragments** across components.

- The **uncatchable failure** is a subagent returning a **confident, well-formed, wrong result** that the orchestrator **cannot independently doubt**. The documented countermeasure is a **verification wave** that **re-derives claims from source** and **defaults to refuted when uncertain**.

- A single component must **own sequencing**. That one owner is what turns ordering and halt-on-exception into **design guarantees instead of behaviours**, and it is what makes the run auditable end to end.

- A serious orchestration **isolates failures** so one broken worker **degrades to an error string**, bounds each call with a **per-request timeout**, and **journals work content-addressed** by a hash of each subagent's prompt so a **restart recomputes only what is unfinished**.

- When several tool calls arrive in one turn, **every call needs a result in the next message**, **matched by identifier**, including an **error result with a brief explanation** for any call you chose not to run.

- A server tool call whose identifier has **no matching result block** is unfinished, and there is **no other marker** for it. Detect it by **checking identifiers**. A paused turn at a **server-side iteration limit, default 10**, is handled by **continuing the conversation**, not by treating it as an error.

- A task budget is a **soft hint the model may exceed**, shown as a countdown it paces against; the **enforced ceiling** is the request's **maximum token setting**. A scenario needing a hard limit needs that setting.

- **Tool-selection errors scale** with the number of **unrelated domains sharing one context**, so the only durable fix **reduces what a single agent chooses among**. Longer descriptions, more examples and routing instructions all leave the crowded surface intact.

- Extracting a decision from model output with a regular expression means the **decision should have been a tool call**. **Tool use is a contract**: you specify the operations and the shape of inputs and outputs, the model decides when to call them.